

OS Fingerprinting: Passive, Active & TCP/IP Stacks

Active OS detection (nmap)

```
sudo nmap -O --osscan-guess 10.0.0.20
```

nmap sends unusual probes; the TCP/IP stack's quirks (TTL, window size, IPID, TCP options) are a unique fingerprint.

Passive fingerprinting (p0f)

```
sudo p0f -i eth0
# captures SYN and SYN/ACK to identify remote OS by packet fields
```

p0f is invisible – it never sends anything.

What the fingerprint reveals

OS	Default TTL	TCP Window	Common quirks
Linux	64	~64k, varies	Window = 1460*n
Windows	128	~8k-16k	Windows-scaling options differ
macOS	64	65k+	Option order unique

Why it matters

- Lateral movement: if you know the target's OS, you pick the right exploit
- Evasion: banners can lie; OS fingerprinting is harder to spoof than a service banner

Fingerprint confusion / defense

Host-based firewalls, proxies, and NAT mangle TTLs

Normalization in IDS can alter observable windows

False positives: Windows-like fingerprints on a Linux behind certain proxies

Professional rule: trust service versions + OS fingerprint together, not either in isolation. The service version is your exploitation lead; the OS is your payload selector.

[← Service Enumeration Deep](#)

[From Vulnerability Scan to Exp...](#)